

多流调度机制重构方案

目录

- 一、背景
- 二、升级改造方案
 - （一）算子类型划分
 - （二）依赖类型划分
 - （三）线程分发与调度
 - （四）多流同步
- 三、一些问题讨论
 - （一）如何知道一个算子是cuda同步还是cuda异步呢？
 - （二）CUDA event与op相绑定，与原先绑定到variable上相比，有什么好处？

一、背景

新执行器最初设计的多流调度机制，只考虑了较为简单的调度场景，未全面考虑多线程结合多流在各种复杂组合场景下算子的调度逻辑，在实际应用时既存在较多的功能、性能漏洞，也难以满足自定义stream、计算通信多流、跨step overlap等复杂调度场景下的优化需求。

在新执行器的现有设计中，算子在调度时被根据其执行place划分成kQueueSync和kQueueAsync两种类型，**CPUPlace**的算子对应**kQueueSync**，**GPU Place**的算子对应**kQueueAsync**。

在调度线程层面，新执行器设计了**HostThread**和**DeviceThread**两个线程池，前者用于调度kQueueSync的算子，后者用于调度kQueueAsync的算子。

为了在多流场景下正确进行数据同步，一个算子的后继算子又被进一步划分为direct_run、synchornize_run和event_run三种类型，**direct_run**表示“可以在当前线程直接拉起，不需要切换线程”，**synchnrize_run**表示该后继算子“是个昂贵的同步等待操作，建议放到单独一个线程执行”，**event_run**表示该后续算子与当前算子“不属于一个stream流，需要额外处理”（相关设计描述见PR#35886）。在GPU训练场景下，当前的direct_run包含以下四种情况：

1. 具有相同DeviceContext的两个依赖算子为direct_run，即CPU算子到CPU算子（单流）、GPU算子到GPU算子
2. CPU算子的所有下游算子均为direct_run
3. D2H到CPU为direct_run
4. CPU到H2D为direct_run

除了以上四种情况，剩下的GPU算子到CPU算子被归为SyncRun，GPU算子到GPU算子（跨流）被归为event_run。

在实际做多线程调度时，对于kQueueAsync类型的算子，其下游的synchornize_run算子会被AddTask分发到其对应的线程池中，而direct_run和event_run类型的算子则会保持在当前线程中执行。对于kQueueSync类型的算子，其下游的event_run算子会被AddTask分发到其对应的线程池中，而synchornize_run和direct_run的算子会保留一个在当前线程中执行，其它算子也通过AddTask分发到其对应的线程池中。

在多流同步方面，对于synchornize_run和event_run的上下游依赖，会在所有既是上游算子输出、又是下游算子输入的变量上绑定对应的**CUDA event**，通过在上游算子调度后record event，下游算子调度之前wait event，来达到跨流同步的目的。而对于direct_run的算子则不做event同步。

这套机制最初设计时对多流场景的前提假定约束较强，后续执行器在扩展各种复杂多流场景时，逐渐暴露出许多问题。如：

1. 已有设计假定CPUPlace和kQueueSync、GPUPlace和kQueueAsync一一对应，但实际场景中存在将GPUPlace对应到kQueueSync的需求。比如coalease_tensor、shape等算子虽是GPU place，但其计算逻辑并不涉及launch CUDA kernel，将其对应到kQueueSync而不阻塞DeviceThread线程池，可提升调度性能。
2. 已有设计对direct_run、synchornize_run和event_run的划分是不清晰且不全面的。在划分逻辑上杂糅了“指导线程池分发操作”和“指导流同步操作”两个不同的维度，这导致在实际实现时存在较多调度逻辑漏洞，难以应对各种复杂的多流调度场景。如有些算子可能不需要切换线程（direct_run），但却需要跨流同步（event_run）；有些算子需要切换线程（synchornize_run），但并不需要做同步等。针对这块虽已有多次打补丁式的bug修复（PR47606, PR47843），但不清晰和不全面的逻辑划分始终留有安全隐患，很难通过一次次地填坑修复将其变得全面和鲁棒。
3. 已有设计假定跨流同步仅发生在上游算子有数据输出到下游算子的情况，但实际中跨流的上游算子可能并不存在数据依赖。这种假定存在较多的问题，具体分析见后续新设计方案中“多流同步”部分的阐述。

因此，有必要对新执行器的多流调度机制进行重新梳理和升级，以适应后续自动并行大模型极致优化场景下复杂的多流调度需求。

二、升级改造方案

（一）算子类型划分

保持原有kQueueSync和kQueueAsync两种算子类型不变，但不再仅通过place来做区分。其中kQueueSync表示该算子的所有运行逻辑均为CUDA同步的，而kQueueAsync表示该算子的运行逻辑中存在 CUDA异步操作。如果一个GPU算子没有CUDA操作或仅调用了一些同步的CUDA API，则该算子类型可以为kQueueSync（实际上现在新执行器中已经这么做了）。

算子类型直接影响线程池的分发，kQueueSync和kQueueAsync类型严格地对应分发到HostThread和DeviceThread线程池中进行调度。

（二）依赖类型划分

上下游算子的依赖关系，保留direct_run、synchornize_run和event_run三种类型划分，但对这三种类型所表示的语义和划分原则进行进一步的清晰化和规范化，以避免模糊的设计在实现阶段引入较多代码漏洞。依赖关系的类型根据上下游算子是否需要做同步操作来进行划分，其中direct_run表示上下游算子之间在主机端和设备端都不需要做同步操作，即上游算子调度完成后可立即调度下游算子；synchornize_run表示下游算子调度时需要做主机端同步，即下游算子必须等待上游算子的CUDA异步操作执行完成方可被调度；event_run表示下游算子调度时需要做设备端同步，即上游算子不阻塞下游算子的CPU执行，但下游算子的CUDA异步操作需要等待上游算子的CUDA异步操作完成。

三种依赖关系类型的划分仅用于指导算子间的多流同步操作，不考虑任何线程调度的逻辑，也不对算子的线程分发做任何约束。

基于以上划分原则，我们进一步梳理不同算子间上下游依赖所对应的类型。在不约束线程分发的前提下，算子执行的place有CPU和GPU两种类型（关联计算设备），算子类型有kQueueSync和kQueueAsync两种类型（关联调度线程），若将计算设备和调度线程两个维度共同考虑，则一个算子可被划分成以下三种情况（CPU + kQueueAsync的情况不存在）：

- ① CPU + kQueueSync
- ② GPU + kQueueSync
- ③ GPU + kQueueAsync

对于上下游不同情况的算子，其对应的依赖类型如下：

		下游 ①	下游 ②	下游 ③
1				
2	上游 ①	direct	direct	direct
3	上游 ②	direct	direct	direct
4	上游 ③	sync	同流direct、跨流event	同流direct、跨流event

即③-->①为synchornize_run，③-->②和③-->③若跨流则为event_run，其它均为direct_run。

新执行器已有方案中direct_run还对memcpy的算子做了特殊判断，在新的划分方式里，memcpy的算子遵循和其它算子一样的划分原则，不需要特殊处理。

(三) 线程分发与调度

在上述依赖类型划分体系下，线程分发逻辑保持原有的设计思想不变，即对于kQueueAsync类型的算子，将其下游的synchornize_run算子分发到对应的线程池中，而direct_run和event_run类型的算子则保持在当前线程中执行。对于kQueueSync类型的算子，将其下游的event_run算子AddTask分发到对应的线程池中，而synchornize_run和direct_run的算子会保留一个在当前线程中执行，其它算子也通过AddTask分发到对应的线程池中。

唯一需要改动的是，所有保留在当前线程执行的算子，必须是类型相同的算子。即kQueueAsync类型的算子保留的下游算子只允许是kQueueAsync类型，kQueueSync类型的算子保留的下游算子只允许是kQueueSync类型。

(四) 多流同步

执行器的多流同步通过插入CUDA event来实现，多流同步的逻辑应尽量做到对event的插入和同步操作不重不漏，多了会产生较大的性能开销，少了则会影响计算结果的正确性。

已有的逻辑对synchornize_run类型的上下游依赖会调用CUDAStreamSynchronize进行同步，对于event_run类型的上下游依赖会调用CUDAStreamEventWait进行同步，这一点在设计思想上本身是没有问题的，只不过受限于依赖类型划分的不准确，在实际应用时才产生了较多的问题。在重新梳理上下游依赖类型后，可继续沿用已有做法对不同类型依赖采用对应的CUDA流同步方法。

新的升级方案相比执行器中已有的逻辑，着重在上下游是否有实际数据依赖的考虑。已有设计中对用于在不同算子间做同步关联的CUDA event是锚定在Variable上的，在需要做流同步的上下游算子中，每当上游算子有一个变量输出到下游算子，就会在这个关联的变量上绑定一个做同步的event。如果上游算子没有变量输出到下游算子，则不做同步操作。当前上游算子和下游算子没有变量关联的情况主要有上下游之间只存在控制依赖以及下游算子注册了no_need_buffer两种场景。

这种流同步的逻辑是有欠考虑的。首先，数据依赖并非只有变量同时是上游的输出和下游的输入这一种情况，还包含“同时是上游的输出和下游的输出”以及“同时是上游的输入和下游的输出”，另外两种情况不做跨流同步，是无法保证结果正确性的；其次，一对上下游算子之间只需要做一次同步，在有多输入多输出的情况下，给每个变量都锚定一个CUDA event，是冗余的；最后，对于需要做流同步的上下游算子，即使它们之间没有数据依赖，需要做流同步的条件也是客观存在的，在没有数据依赖时直接跳过同步操作，也是无法保证结果正确性的。

基于以上考虑，重新设计的多流同步逻辑如下：

1. **direct_run**不需要做同步，**synchronize_run**和**event_run**必须做同步。
2. 用于同步的**CUDA event**不与**variable**相绑定，而是与**op**相绑定。每个Instruction结构中原来维护了两个event列表input_events_和output_events_，分别与锚定的input和output变量相对应。新的设计方案中input_events_将和具体的算子做锚定，当一个算子需要和同一个流上的多个算子做同步时，只需要维护最后一个调度的算子Record的event。而对于output_events_，每个算子只需要维护一个即可，上游算子调度完成后在其自身的执行流上record该event，下游算子调度之前wait该流对应的event，即可实现同步。这种方式可以在保证同步结果正确性的前提下，尽可能地降低event操作的次数。
3. 没有数据关联的上下游算子，跳过跨流同步。但若一个需要建立同步关系的下游算子由于没有数据关联而被跳过，则需要递归地对该下游算子的所有下游算子建立同步关联。数据关联包括写后读、读后写和同写三种情况。

三、一些问题讨论

（一）如何知道一个算子是cuda同步还是cuda异步呢？

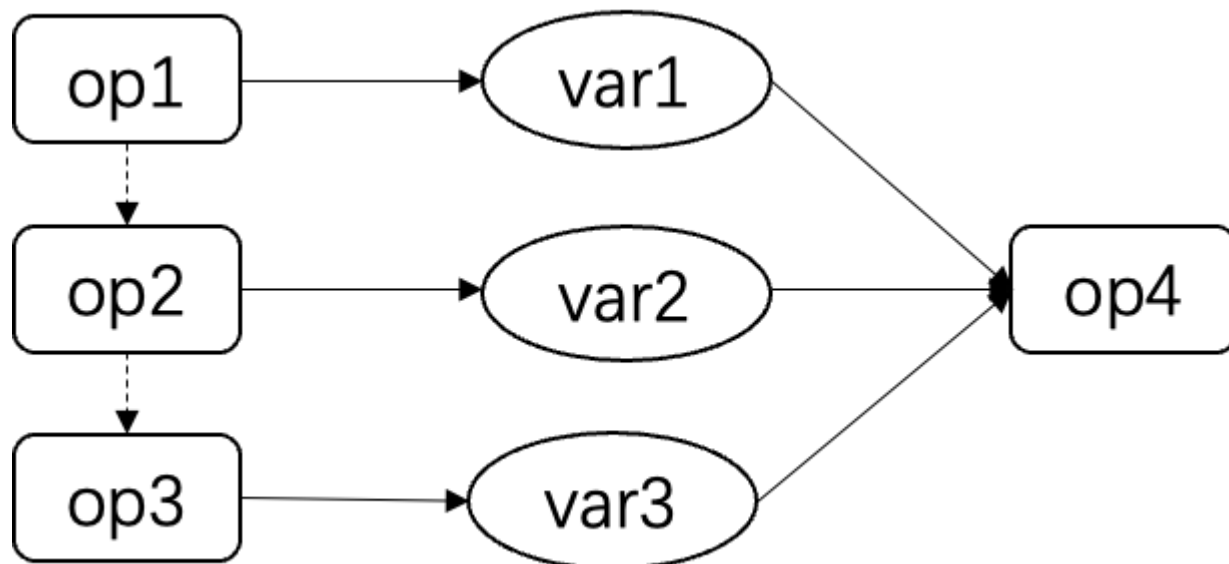
现在执行器中主要通过手动指定的方式，后续可以再考虑一些更自动的方法。为了性能优化，现在已经被指定到kQueueSync的GPU算子包括coalease_tensor、shape、memcpy等。

（二）CUDA event与op相绑定，与原先绑定到variable上相比，有什么好处？

最优情况下，一对上下游算子只需要使用一个event做同步，且相同流的event应尽量能做到复用。与variable相绑定的方式，在算子有多个输入或输出variable的情况下，将导致大量的冗余event操作，从而加大调度开销。

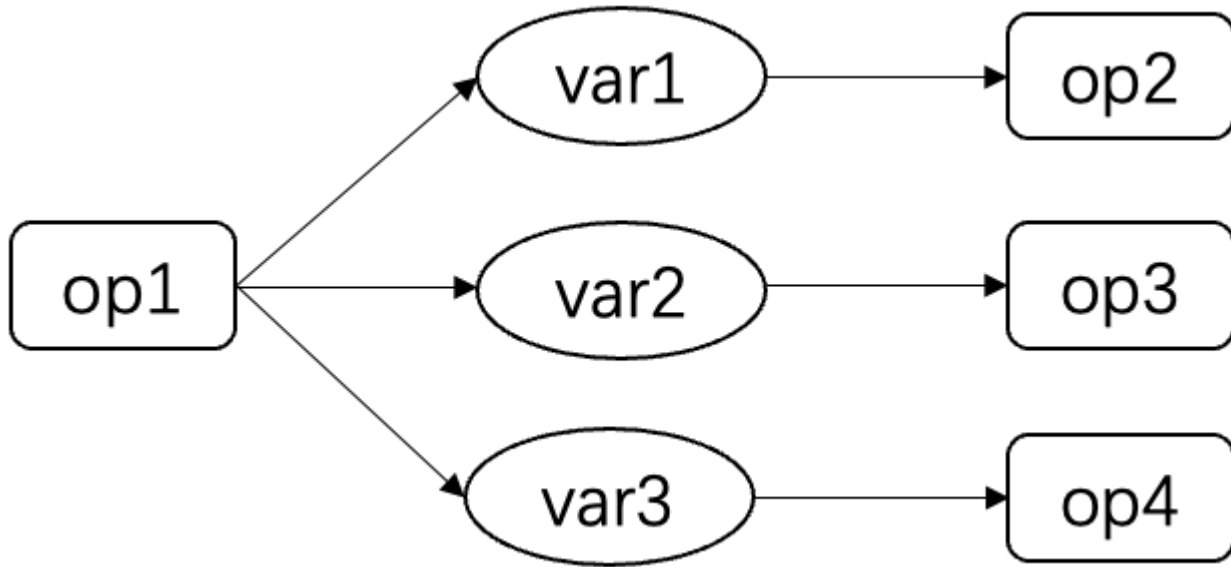
以下分别从多输入和多输出两个维度举例说明两种方式的对比：

1. 算子有多输入的情况



以上图为例，op4有三个输入变量，分别来自op1、op2和op3，op1、op2和op3在调度上有先后顺序（这种先后顺序的约束可能来自依赖边，也可能来自op的调度优先级）。若op1、op2和op3处于同一个流，op4处于另一个流。当event与variable相绑定时，op4调度之前需要等待与var1、var2和var3相对应的3个event；当event与op相绑定时，由于op1、op2和op3处于相同流且有调度先后顺序，因此op4可以只等待最后一个调度的op3对应的event，而不需要在op1和op2之间建立event同步。

2. 算子有多输出的情况



反过来，若如上图中，op1与三个变量分别输出到op2、op3和op4，op1在一个流，op2、op3和op4在另一个流。当event与variable相绑定时，op1在调度后需要在对应的流上record与var1、var2和var3相对应的3个不同的event；当event与op相绑定时，op1在调度后只需要record与其自身对应的1个event，op2、op3和op4均等待相同的event即可。

总而言之，event与variable相绑定只有在所有同步算子均为单输入单输出的情况下，才能保证较优的event开销。而在分布式多流的场景下，由于fuse操作的存在，多输入多输出是非常常见的场景，为了极致的调度性能，需要将其升级成与op绑定。